



## Fundamentals of Big Data Analytics (CS4109)

---

# Real-Time Crime Analytics & Intelligent Alert System

---

**Deadline: 11-05-26 11:59PM**

### Course Team

---

#### Instructor:

- Ms. Kainat Iqbal  
[kainat.iqbal@isb.nu.edu.pk](mailto:kainat.iqbal@isb.nu.edu.pk)
- Dr. Asif Muhammad  
[asif.muhammad@isb.nu.edu.pk](mailto:asif.muhammad@isb.nu.edu.pk)

#### Teaching Assistants:

- Moaz Murtaza  
[i221902@nu.edu.pk](mailto:i221902@nu.edu.pk)
- Bilal Bashir  
[i221901@nu.edu.pk](mailto:i221901@nu.edu.pk)

---

## Contents

---

|           |                                                  |          |
|-----------|--------------------------------------------------|----------|
| <b>1</b>  | <b>Introduction</b>                              | <b>3</b> |
| <b>2</b>  | <b>Learning Objectives</b>                       | <b>3</b> |
| <b>3</b>  | <b>Dataset Description</b>                       | <b>4</b> |
| 3.1       | Key Schema Fields . . . . .                      | 4        |
| <b>4</b>  | <b>Problem Statement</b>                         | <b>4</b> |
| <b>5</b>  | <b>System Architecture</b>                       | <b>5</b> |
| 5.1       | Batch Layer — Apache Spark . . . . .             | 5        |
| 5.2       | Speed Layer — Kafka + Apache Storm . . . . .     | 5        |
| 5.3       | Serving Layer — PostgreSQL & MongoDB . . . . .   | 6        |
| 5.4       | Infrastructure . . . . .                         | 6        |
| <b>6</b>  | <b>Required Streaming Analytics</b>              | <b>6</b> |
| <b>7</b>  | <b>Additional Analysis</b>                       | <b>7</b> |
| 7.1       | Crime Trend Analysis . . . . .                   | 7        |
| 7.2       | Arrest Rate Analysis . . . . .                   | 7        |
| 7.3       | Violence and Gunshot Analysis . . . . .          | 7        |
| 7.4       | Sex Offender Proximity Analysis . . . . .        | 7        |
| 7.5       | Geospatial Hotspot Detection (K-Means) . . . . . | 7        |
| 7.6       | Cross-Dataset Correlation . . . . .              | 8        |
| <b>8</b>  | <b>Execution Expectations</b>                    | <b>8</b> |
| <b>9</b>  | <b>Academic Integrity</b>                        | <b>8</b> |
| <b>10</b> | <b>Deliverables</b>                              | <b>9</b> |

## 1. Introduction

---

Urban crime analysis presents a critical challenge for public safety agencies. Traditional batch reporting systems are insufficient for time-sensitive decision-making. Modern cities require intelligent, real-time analytics pipelines capable of detecting anomalies, identifying hotspots, and dispatching alerts as incidents unfold.

In this project, you will build a **Real-Time Crime Analytics and Intelligent Alert System** powered by the City of Chicago's publicly available public safety datasets. Using a **Lambda Architecture**, your system will simultaneously support:

- **Batch processing** — historical trend analysis, machine learning, and cross-dataset joins using Apache Spark.
- **Stream processing** — real-time event ingestion, windowed anomaly detection, and alert generation using Apache Kafka and Apache Storm.
- **Serving layer** — structured storage in PostgreSQL and MongoDB, with an optional real-time Streamlit dashboard.

This project mirrors production-grade big data pipelines used by law enforcement analytics platforms, smart city initiatives, and public safety research centres.

## 2. Learning Objectives

---

Upon successful completion of this project, students will be able to:

1. Design and implement a **Lambda Architecture** combining batch and speed layers.
2. Ingest and preprocess multi-schema heterogeneous CSV datasets using **Apache Spark**.
3. Define and enforce explicit **StructType schemas** for real-world, dirty data.
4. Publish and consume streaming events using **Apache Kafka** producers and topics.
5. Implement multi-bolt **Apache Storm topologies** with windowing, filtering, and alert logic.
6. Apply **unsupervised machine learning** (K-Means clustering) for geospatial hotspot detection.
7. Perform **cross-dataset joins and correlations** across heterogeneous public safety schemas.
8. Persist analytics results to **PostgreSQL** and **MongoDB** and expose them via a dashboard.
9. Containerise and orchestrate all system components using **Docker Compose**.
10. Communicate system design clearly through structured **technical documentation**.

### 3. Dataset Description

All datasets are sourced from the **City of Chicago Open Data Portal**. You are required to download and use all five datasets listed below.

| Dataset                   | Description                                                                                                                            | Link                         |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------|------------------------------|
| <b>Crime Data</b>         | All reported criminal incidents in Chicago from 2001 to the present. Includes crime type, location, date, arrest status, and district. | <a href="#">View Dataset</a> |
| <b>Police Stations</b>    | Locations, contact details, and coordinates of all Chicago Police Department district stations.                                        | <a href="#">View Dataset</a> |
| <b>Arrests</b>            | Individual arrest records including charges, charge classifications, and statutory references.                                         | <a href="#">View Dataset</a> |
| <b>Violence Reduction</b> | Victims of homicides and non-fatal shootings. Includes FBI codes, victim demographics, street outreach, and gunshot indicators.        | <a href="#">View Dataset</a> |
| <b>Sex Offenders</b>      | Registered sex offenders including block-level address, demographics, and victim minor flag.                                           | <a href="#">View Dataset</a> |

#### 3.1. Key Schema Fields

The table below summarises the most important fields you will work with across datasets. You must handle all field-level nulls, type mismatches, and naming inconsistencies.

| Dataset                | Key Columns                                                                                                                                                                              |
|------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Sex Offenders</b>   | LAST, FIRST, BLOCK, GENDER, RACE, BIRTH_DATE, HEIGHT, WEIGHT, VICTIM_MINOR                                                                                                               |
| <b>Violence</b>        | SEX, RACE, VICTIMIZATION_FBI_CD, INCIDENT_FBI_CD, HOMICIDE_VICTIM_*, MONTH, DAY_OF_WEEK, CASE_NUMBER, DATE, BLOCK, GUNSHOT_INJURY_I, ZIP_CODE, WARD, COMMUNITY_AREA, DISTRICT, BEAT, AGE |
| <b>Arrests</b>         | CB_NO, CASE_NUMBER, ARREST_DATE, RACE, CHARGE_1..4_STATUTE, CHARGE_1..4_DESCRIPTION, CHARGE_1..4_TYPE, CHARGE_1..4_CLASS                                                                 |
| <b>Police Stations</b> | DISTRICT, DISTRICT_NAME, ADDRESS, CITY, STATE, ZIP, PHONE, X_COORDINATE, Y_COORDINATE, LATITUDE, LONGITUDE                                                                               |

### 4. Problem Statement

Chicago's public safety ecosystem generates thousands of records daily across crime reports, arrest logs, violence incidents, and sex offender registrations. These datasets are siloed, heterogeneous in schema, and currently processed in isolation using static batch tools.

Your task is to **design and implement a unified, real-time crime analytics platform** that:

- **Ingests** all five Chicago public safety datasets.
- **Streams** crime events row-by-row through a Kafka producer simulating live incident feeds.
- **Detects anomalies** in real time using Apache Storm — specifically, unusual spikes in crime counts within sliding time windows.
- **Generates intelligent alerts** when district-level thresholds are breached, routing them to persistent storage.
- **Performs batch analytics** using PySpark to uncover temporal patterns, arrest rates, cross-dataset correlations, and geospatial crime hotspots.
- **Clusters crime hotspots** using K-Means to assist in police resource allocation.
- **Persists** all results — alerts, trends, hotspots, correlations — to PostgreSQL and MongoDB.
- **Visualises** key metrics on a live Streamlit dashboard (optional but recommended).

**Scope Constraint:** You are not expected to integrate with live APIs or deploy to a cloud environment. All streaming must be simulated by replaying CSV rows through your Kafka producer at a controlled rate (e.g., 1 row per second).

## 5. System Architecture

Your implementation must follow the **Lambda Architecture** pattern, composed of three layers:

### 5.1. Batch Layer — Apache Spark

The batch layer reads all five datasets from disk (HDFS or local), enforces explicit schemas, performs preprocessing, computes analytics, and writes results to the serving layer.

- Define **StructType** schemas for all datasets.
- Handle nulls, type casting, and column renaming.
- Compute crime trends (yearly, monthly, hourly, day-of-week).
- Compute arrest rates by crime type, district, and race.
- Join datasets (e.g., crimes  $\bowtie$  arrests on **CASE\_NUMBER**; crimes  $\bowtie$  police stations on **DISTRICT**).
- Run K-Means clustering on geospatial coordinates to detect crime hotspots.
- Compute cross-dataset correlations (e.g., violence rate vs. arrest rate by district).

### 5.2. Speed Layer — Kafka + Apache Storm

The speed layer simulates real-time ingestion by streaming crime records and applies windowed anomaly detection.

- **Kafka Producer:** Reads the Crime dataset row-by-row and publishes each row as a JSON message to the `crime_events` topic at a configurable rate (default: 1 row/second).
- **Storm Topology:** A multi-bolt topology consuming from Kafka:
  - `KafkaSpout` — Reads from the `crime_events` topic.
  - `ParseBolt` — Deserialises and validates each JSON message.
  - `DistrictBolt` — Routes events by police district.
  - `WindowBolt` — Applies a sliding time window (e.g., 5 minutes) to count events per district.
  - `AnomalyBolt` — Flags districts where event counts exceed a configurable threshold.
  - `AlertBolt` — Writes triggered alerts to MongoDB and/or PostgreSQL.

### 5.3. Serving Layer — PostgreSQL & MongoDB

- **PostgreSQL** stores structured analytics outputs: `crime_trends`, `hotspots`, `correlations`, and `alerts`.
- **MongoDB** stores semi-structured documents: raw event logs and alert records with full JSON payloads.

### 5.4. Infrastructure

All components must be containerised and launched with a single `docker-compose up` command. The required services are: Zookeeper, Kafka, Apache Spark, Apache Storm (Nimbus + Supervisor + UI), PostgreSQL, and MongoDB.

#### Recommended Folder Structure:

```
crime_analytics/
  docker/  (docker-compose.yml, Dockerfiles)
  spark/   (schemas/, preprocessing/, analytics/, ml/)
  kafka/   (producer.py)
  storm/   (topology/, spouts/, bolts/)
  dashboard/ (streamlit_app.py)
  data/    (CSV files)
  db/      (postgres_init.sql, mongo_setup.py)
  config/   (config.yaml)
  requirements.txt, README.md
```

## 6. Required Streaming Analytics

The following streaming analytics components are **mandatory** and will be assessed individually.

1. **Kafka Producer (Crime Simulator):** Streams the Crime dataset row-by-row as JSON to

the `crime_events` Kafka topic. The publication rate must be configurable via `config.yaml`. Each message must include at minimum: `CASE_NUMBER`, `DATE`, `BLOCK`, `PRIMARY_TYPE`, `DISTRICT`, `ARREST`, `LATITUDE`, `LONGITUDE`.

2. **Parse Bolt:** Deserialises each Kafka message, validates required fields, and emits a structured tuple downstream. Malformed messages must be logged and discarded, not forwarded.
3. **District Bolt:** Groups incoming tuples by police district. Emits district-tagged tuples to the window bolt for per-district counting.
4. **Window Bolt (Sliding Window):** Maintains a sliding count window (configurable size and slide interval). Emits per-district crime counts at each slide interval.
5. **Anomaly Bolt:** Compares each district's window count against a configurable baseline threshold. Emits an anomaly tuple when the threshold is exceeded, including district ID, window count, threshold, and timestamp.
6. **Alert Bolt:** Consumes anomaly tuples and persists structured alert records to both MongoDB (`alert_logs` collection) and PostgreSQL (`alerts` table). Each alert must include: district, timestamp, event count, threshold, and alert severity level.

## 7. Additional Analysis

---

The following batch analytics components are also **required** as part of the Spark batch layer.

### 7.1. Crime Trend Analysis

Compute and store aggregated crime counts grouped by: year, month, day of week, and hour of day. These trends must be written to the `crime_trends` table in PostgreSQL.

### 7.2. Arrest Rate Analysis

Calculate the arrest rate (arrests/crimes) broken down by: primary crime type, police district, and race. Join the Crime and Arrests datasets on `CASE_NUMBER`. Identify the top 10 crime types by arrest rate.

### 7.3. Violence and Gunshot Analysis

From the Violence dataset, compute: (a) total homicides vs. non-fatal shootings by month and district; (b) proportion of incidents involving confirmed gunshot injuries (`GUNSHOT_INJURY_I = 'Y'`); (c) top community areas by violence incidence.

### 7.4. Sex Offender Proximity Analysis

Join the Sex Offenders dataset with Police Stations on district. Identify districts with the highest density of registered offenders. Flag records where `VICTIM_MINOR = 'Y'` for priority reporting.

### 7.5. Geospatial Hotspot Detection (K-Means)

Apply K-Means clustering ( $k = 10$ ) to crime event coordinates (`LATITUDE`, `LONGITUDE`) using PySpark MLlib. Store cluster centroids and labels in the `hotspots` PostgreSQL table. Visualise

the clusters on a map (static plot or dashboard widget).

## 7.6. Cross-Dataset Correlation

Compute at least two cross-dataset correlations, for example:

- Violence rate vs. total arrest rate, grouped by district.
- Sex offender density vs. crime rate, grouped by community area.

Results must be written to the `correlations` table in PostgreSQL.

## 8. Execution Expectations

1. **Single-command launch:** Running `docker-compose up` must start all infrastructure services (Kafka, Zookeeper, Storm, Spark, PostgreSQL, MongoDB) without manual intervention.
2. **Reproducibility:** The Kafka producer, Storm topology, and Spark jobs must all be executable with a single script or `Makefile` target after containers are running.
3. **Schema enforcement:** All Spark DataFrames must use explicit `StructType` schemas. Schema inference (`inferSchema=True`) is not permitted in the final submission.
4. **Error handling:** The Storm topology must handle malformed messages without crashing. The Kafka producer must handle CSV read errors gracefully.
5. **Configuration:** All configurable parameters — Kafka broker address, topic name, window size, anomaly threshold, database credentials — must be externalised in `config/config.yaml`.
6. **Data volume:** You may use a representative sample of each dataset (minimum 50,000 rows for Crime, 10,000 rows for others) if full datasets are too large for local execution.
7. **Dashboard (optional):** A Streamlit dashboard displaying live alerts, trend charts, and a hotspot map will earn bonus marks.

## 9. Academic Integrity

**This is a group project. All work submitted must be original and produced by your group.**

- Group of 2 People is allowed. Fill your group name (Any) with details for groups student.
- You may use publicly available documentation, tutorials, and Stack Overflow answers for reference, but all code submitted must be written by your group.
- Use of AI code generation tools (e.g., GitHub Copilot, ChatGPT) must be **disclosed** in your report. Attaching a screenshot of prompt can get you some extra marks as well also Undisclosed use will be treated as academic dishonesty.



- Sharing code between groups in any direction is strictly prohibited and will result in a zero for all parties involved.
- Plagiarism detection tools will be applied to all submissions.
- Each member must be able to explain any part of the submitted code during a viva or demo session.

## 10. Deliverables

Submit all materials as a single compressed archive (.zip) via the course portal by the stated deadline. The archive must contain:

| # | Deliverable            | Description                                                                                                                                                                | Weight |
|---|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| 1 | Source Code            | Complete project source (all Spark, Kafka, Storm, dashboard, and DB scripts). Must match the required folder structure.                                                    | 60%    |
| 2 | Docker Setup           | <code>docker-compose.yml</code> and all Dockerfiles. Must launch the full system with <code>docker-compose up</code> .                                                     | 5%     |
| 3 | Technical Report       | PDF report ( $\leq 20$ pages) covering: system design, schema descriptions, analytics methodology, ML approach, results (tables/charts), and challenges faced.             | 15%    |
| 4 | Demo                   | A 5–8 minute screen-recorded demonstration and demo showing: Kafka streaming, Storm topology in Storm UI, Spark job completion, and query results from PostgreSQL/MongoDB. | 15%    |
| 5 | Contribution Statement | Signed document listing each member's contribution by component (percentage breakdown).                                                                                    | 5%     |

### Dataset Links (for Reference)

- **Crime Data (2001–Present):**

[https://data.cityofchicago.org/Public-Safety/Crimes-2001-to-Present/ijzp-q8t2/about\\_data](https://data.cityofchicago.org/Public-Safety/Crimes-2001-to-Present/ijzp-q8t2/about_data)

- **Police Stations:**

[https://data.cityofchicago.org/Public-Safety/Police-Stations/z8bn-74gv/about\\_data](https://data.cityofchicago.org/Public-Safety/Police-Stations/z8bn-74gv/about_data)

- **Arrests:**

[https://data.cityofchicago.org/Public-Safety/Arrests/dpt3-jri9/about\\_data](https://data.cityofchicago.org/Public-Safety/Arrests/dpt3-jri9/about_data)

- **Violence Reduction (Homicides & Non-Fatal Shootings):**

[https://data.cityofchicago.org/Public-Safety/Violence-Reduction-Victims-of-Homicides-and-gumc-mgzz/about\\_data](https://data.cityofchicago.org/Public-Safety/Violence-Reduction-Victims-of-Homicides-and-gumc-mgzz/about_data)

- **Sex Offenders:**

[https://data.cityofchicago.org/Public-Safety/Sex-Offenders/vc9r-bqvy/about\\_data](https://data.cityofchicago.org/Public-Safety/Sex-Offenders/vc9r-bqvy/about_data)

---

*Good luck. Build something that matters.*